

Plan de Pruebas Unitarias - Restaurante Bambú

Sistema de Pedidos en Línea

1. Introducción

Este documento presenta el **Plan de Pruebas Unitarias** para el proyecto Restaurante Bambú, siguiendo la metodología de Pressman (Capítulo 17, sección 17.4: "Pruebas de Unidad", páginas 391-395).

1.1 Definición (Pressman, p. 391)

"Las pruebas de unidad centran el proceso de verificación en la menor unidad del diseño del software: el componente de software o módulo."

1.2 Objetivo

Probar cada componente de manera individual utilizando la descripción del diseño a nivel de componentes como guía. Se prueban:

- Interfaz del módulo
- Estructuras de datos locales
- Condiciones límite
- Caminos independientes
- Caminos de manejo de errores

2. Enfoque: Caja Blanca (White Box Testing)

Definición (Pressman, p. 407):

"Las pruebas de caja blanca derivan casos de prueba a partir del conocimiento de la estructura interna y la implementación del componente."

Para este proyecto, aplicaremos:

- **Prueba del camino básico:** Verificar todos los caminos independientes.
- **Prueba de condiciones:** Validar condiciones lógicas.
- **Prueba de bucles:** Verificar estructuras iterativas (si aplica).

3. Componentes a Probar

3.1 Modelos de Mongoose

Modelo	Archivo	Responsabilidad	Prioridad
User	<code>src/models/User.js</code>	Gestión de usuarios, validación de email único	Alta
MenuItem	<code>src/models/MenuItem.js</code>	Productos del menú, esquema de extras y tamaños	Alta
Order	<code>src/models/Order.js</code>	Pedidos, estado de pago	Alta
UserInfo	<code>src/models/UserInfo.js</code>	Información de perfil del usuario	Media
Category	<code>src/models/Category.js</code>	Categorías de productos	Media

3.2 Estrategia de Prueba para Modelos

Objetivo: Validar esquemas Mongoose sin necesidad de conexión a base de datos.

Técnicas:

1. **Validación de esquema:** Usar `validateSync()` para verificar reglas.
2. **Prueba de valores por defecto:** Verificar campos con valores default.
3. **Prueba de tipos de datos:** Verificar que se rechacen tipos incorrectos.
4. **Prueba de campos requeridos:** Verificar que falten campos obligatorios lance errores.

4. Casos de Prueba Diseñados

Modelo User (10 casos)

- Validación de email requerido
- Validación de email único (requiere BD mock)
- Validación de formato de email
- Valor por defecto de timestamps
- Rechazo de tipos incorrectos

Modelo MenuItem (12 casos)

- Validación de estructura de sizes
- Validación de estructura de extraIngredientPrices
- Validación de referencia a Category (ObjectId)
- Validación de basePrice como número
- Valores por defecto

Modelo Order (8 casos)

- Valor por defecto de `paid: false`
- Validación de userEmail
- Validación de cartProducts como Object

- Timestamps automáticos

Total: ~30 casos de prueba unitarios

5. Herramientas

Herramienta	Versión	Propósito
Jest	^29.0.0	Framework de testing (ya instalado)
Mongoose	^7.6.3	Para pruebas de esquema

No se requiere: MongoDB real (usaremos validación offline de esquemas)

6. Configuración

6.1 Archivo de Configuración

jest.config.unit.js:

```
module.exports = {
  testEnvironment: "node",
  testMatch: ["**/tests/unit/**/*.test.js"],
  collectCoverageFrom: ["src/models/**/*.js", "src/libs/**/*.js"],
};
```

6.2 Script en package.json

```
{
  "scripts": {
    "test:unit": "jest --config=jest.config.unit.js",
    "test:unit:watch": "jest --config=jest.config.unit.js --watch",
    "test:unit:coverage": "jest --config=jest.config.unit.js --coverage"
  }
}
```

7. Estructura de Archivos

```
restaurante-bambu/
├── testing_unitarias/          # Documentación
└── README.md
```

```

├── plan_pruebas_unitarias.md    (este archivo)
├── casos_pruebas_unitarias_modelos.md
└── tests/
    ├── unit/                   # Scripts ejecutables
    │   └── models/
    │       ├── User.test.js
    │       ├── MenuItem.test.js
    │       └── Order.test.js

```

8. Ejecución

```

# Todos los tests unitarios
npm run test:unit

# Con cobertura
npm run test:unit:coverage

# Modo watch (desarrollo)
npm run test:unit:watch

```

9. Criterios de Éxito

- ✓ **100% de tests pasan**
- ✓ **Cobertura de líneas:** ≥90% en modelos
- ✓ **Cobertura de ramas:** ≥80% en validaciones
- ✓ **Tiempo de ejecución:** < 5 segundos (son tests rápidos sin BD)

10. Diferencias con Otros Tipos de Pruebas

Aspecto	Unidad	Integración	Caja Negra
Alcance	Módulo individual	Módulos combinados	Sistema completo
Conocimiento	Código interno (White Box)	Interfaces	Requisitos (Black Box)
Velocidad	Muy rápida (ms)	Rápida (100-500ms)	Variable
Dependencias	Ninguna (aislado)	Reales o mocks	Sistema real
Ejemplo	Validación de esquema User	POST /api/register crea en BD	Usuario puede registrarse

11. Referencias

- **Pressman, R. S.** (2010). *Ingeniería del Software: Un Enfoque Práctico* (7ª ed.). McGraw-Hill.
 - Sección 17.4: Pruebas de Unidad (pp. 391-395)
 - Sección 18.1: Pruebas de Caja Blanca (pp. 407-410)
-

Elaborado por: Equipo de QA - Restaurante Bambú

Fecha: 26 de noviembre de 2024

Versión: 1.0

Estado: Listo para Implementación